Práctica 4: Gestión de imágenes y contenedores

Materia:

Sistemas Distribuidos

Integrantes:

Elkin Yesid Yandun
Julian LeonardoCeron
Brayan Esmid Cruz
Jeison Enriquez

Docente:

LEIDY MARIANI DORADO FLÓREZ

Corporación Universitaria Comfacauca - Unicomfacauca

Popayán

2026

PARTE 1 – Exploración de Imágenes

Actividad 1: Ver imágenes locales

Al ejecutar el comando docker images, se listarán todas las imágenes de Docker que ya tienes en tu sistema.

```
docker images
```

```
PS C:\Users\Elkin yandun> docker images
REPOSITORY                    TAG       IMAGE ID       CREATED        SIZE
nginx                         latest    44d1be91d34f   8 days ago     237MB
mysql                         latest    e5dc14f6e01c   13 days ago    1.25GB
mongo                         7.0       a814f930db8c   4 months ago   1.13GB
patico-app                    latest    3dcfdec19a09   5 months ago   708MB
monitoreo-php                 1.0       e02d7263508e   5 months ago   749MB
ubuntu-set                    latest    83c383abda0e   5 months ago   1.47GB
ubuntu                        latest    353675e2a41b   5 months ago   117MB
gcr.io/k8s-minikube/kicbase   v0.0.48   41454ef774d0   5 months ago   1.84GB
gcr.io/k8s-minikube/kicbase   <none>    7171c97a5162   5 months ago   1.84GB
hello-world                   latest    ef54e839ef54   6 months ago   20.3kB
ubuntu                        18.04     152dc042452c   2 years ago    97.5MB
```

- **¿Qué es el IMAGE ID?** Es un identificador único para cada imagen. Funciona como el “ADN” de la imagen, asegurando que no se confunda con ninguna otra.
- **¿Qué significa TAG?** Es una etiqueta que se le asigna a una imagen para identificar su versión. Por ejemplo, la imagen de mongo tiene los tags 7.0. Si no se especifica, por defecto se usa latest.
- **¿Qué representa SIZE?** Indica el espacio en disco que ocupa la imagen. Este tamaño incluye el sistema operativo base y todas las librerías y archivos que contiene.
- **¿Una imagen se está ejecutando?** No. Una imagen es una plantilla inerte, un paquete que contiene todo lo necesario para ejecutar una aplicación. No consume recursos de CPU o RAM por sí misma, solo espacio de almacenamiento.

Actividad 2: Descargar nuevas imágenes

Voy a descargar una nueva imagen desde Docker Hub, el registro público de imágenes. Usaré postgres como ejemplo con el comando.

```
docker pull postgres
```

```
PS C:\Users\Elkin yandun> docker pull postgres
Using default tag: latest
latest: Pulling from library/postgres
56a239f8dc77: Pull complete
e7dee01ee448: Pull complete
70f3de31936a: Pull complete
be4ad6e904ed: Pull complete
0480b5ae6db9: Pull complete
4e2113a73362: Pull complete
e232269e9149: Pull complete
967e70a4ad93: Pull complete
358db71ef364: Pull complete
1b030cb6bea0: Pull complete
3859b312d253: Pull complete
eea32e6f70e8: Pull complete
Digest: sha256:69e8582b781cb44fa4557b98ed586fe68361e320d9b12f9707494335634f4f3d
Status: Downloaded newer image for postgres:latest
docker.io/library/postgres:latest
PS C:\Users\Elkin yandun> docker images
REPOSITORY                    TAG       IMAGE ID       CREATED        SIZE
postgres                      latest    69e8582b781c   6 days ago     643MB
```

- **¿Qué cambió?** Apareció una nueva imagen en la lista postgres, junto con su IMAGE ID y SIZE.
- **¿El servicio ya está funcionando?** No. Al igual que antes, solo hemos descargado la plantilla. El servicio de postgres aún no está en ejecución.

PARTE 2 – Gestión de Contenedores

Actividad 3: Crear un contenedor

Usaré la imagen nginx que descargamos para crear y ejecutar un contenedor. Un contenedor es una instancia en ejecución de una imagen.

El comando:

```
Docker run -d --name servidor-web -p 8081:80 nginx
```

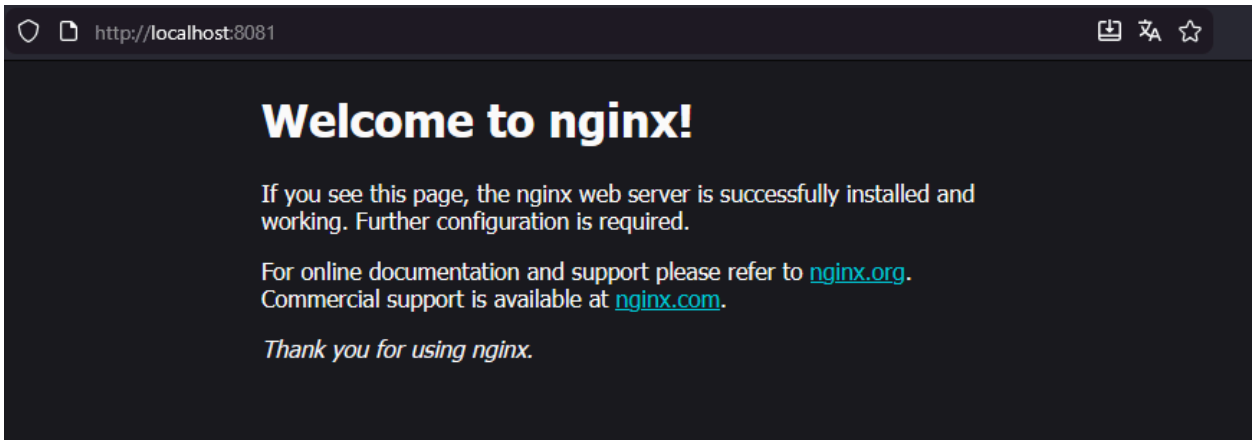
hace lo siguiente: “run” Crea y ejecuta un contenedor. “-d” Lo ejecuta en segundo plano. “--name servidor-web” Le asigna un nombre fácil de recordar. “-p 8081:80” Mapea el puerto 8081 de tu máquina al puerto 80 del contenedor, donde Nginx escucha. “nginx” Es la imagen que se utiliza como base.

Para verificar que está corriendo, usa.

```
docker ps
```

```
PS C:\Users\Elkin yandun> docker run -d --name servidor_web -p 8081:80 nginx
22fab5f5c2b1d3202c0da717b4986cddcc4f9e754f0d9011b11c520a1277476d
PS C:\Users\Elkin yandun> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS
   NAMES
22fab5f5c2b1   nginx     "/docker-entrypoint.…"   9 seconds ago   Up 8 seconds   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
   servidor_web
```

Al abrir tu navegador e ir a http://localhost:8081, se verás la página de bienvenida de Nginx.



Actividad 4: Administrar el contenedor

Un contenedor tiene un ciclo de vida que se puede gestionar:

- **Detener:** “docker stop servidor_web” detiene la ejecución del contenedor. Si intento acceder a http://localhost:8081 ahora, no funcionará.

```
PS C:\Users\Elkin yandun> docker stop servidor_web
servidor_web
```

- **Ver todos:** “docker ps -a” muestra todos los contenedores, incluyendo los que están detenidos. Se mira que servidor-web tiene el estado “Exited”.

```
PS C:\Users\Elkin yandun> docker ps -a
CONTAINER ID   IMAGE                                  COMMAND                  CREATED
  PORTS     NAMES
22fab5f5c2b1   nginx                                  "/docker-entrypoint.…"   3 minutes ago
            servidor_web
eb52428e3be8   nginx                                  "/docker-entrypoint.…"   42 hours ago
            docker-web-1
b9a1ac9c8142   mysql                                  "docker-entrypoint.s…"   42 hours ago
            docker-db-1
c166cdb72b96   nginx                                  "/docker-entrypoint.…"   8 days ago
            relaxed_kepler
39924f3f4296   hello-world                            "/hello"                 8 days ago
            busy_sinoussi
651655c1e5b4   gcr.io/k8s-minikube/kicbase:v0.0.48    "/usr/local/bin/entr…"   5 months ago
            minikube
666e07ca3ff1   b3ff9ad2d855                           "docker-php-entrypoi…"   5 months ago
            monitoreo-app
0bcdb1b3ea70   ubuntu:18.04                           "/bin/bash"              5 months ago
            ubuntuserve
```

- **Reiniciar:** “docker start servidor_web” vuelve a iniciar el contenedor que habías detenido.

```
PS C:\Users\Elkin yandun> docker start servidor_web
servidor_web
```

- **Eliminar:** “docker rm -f servidor_web” elimina el contenedor de forma forzada (-f). Una vez eliminado, no se puede recuperar y se tendrá que crearlo de nuevo con “docker run”.

```
PS C:\Users\Elkin yandun> docker rm -f servidor_web
servidor_web
```

PARTE 3 – Análisis Conceptual

Aquí completamos la tabla comparativa basada en lo que hemos aprendido.

Concepto	Imagen	Contenedor
¿Es ejecutable?	No, es una plantilla.	Sí, es un proceso en ejecución.
¿Ocupa recursos en ejecución?	No (solo almacenamiento en disco).	Sí (CPU, RAM, red).
¿Puede modificarse?	No directamente, se crean nuevas versiones.	Sí, los cambios se guardan en su sistema de archivos.
¿Puede eliminarse?	Sí, con docker rmi.	Sí, con docker rm.

PARTE 4 – Reto en Equipo

Ejemplo usando mongo:

1. **Ejecutar contenedor:** “docker run -d --name mi-base-datos-mongo -p 27017:27017 mongo”

2. **Mostrar que está activo:** “docker ps”

```
PS C:\Users\Elkin yandun> docker run -d --name mi-base-detos-mongo -p 27017:27017 mongo
Unable to find image 'mongo:latest' locally
latest: Pulling from library/mongo
fbfd783f7a59: Pull complete
8d23bbf2b216: Pull complete
427c118b2079: Pull complete
e46eb2190ffc: Pull complete
cea2e6a72e6b: Pull complete
e79e499203c9: Pull complete
01d7766a2e4a: Pull complete
f97b55066391: Pull complete
Digest: sha256:474f5c3bf0e355bb97dafda730e725169a4d51c5578abf7be9ec7ad3fdee4481
Status: Downloaded newer image for mongo:latest
28082004c44bdae045ca83c6f413d229bc4f403645211d3337ba3e529a28f121
PS C:\Users\Elkin yandun> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED              STATUS              POR
                    NAMES
28082004c44b   mongo     "docker-entrypoint.s…"   About a minute ago   Up About a minute   0.0
]:27017->27017/tcp   mi-base-detos-mongo
```

3. **Explicación:**

   - **¿Qué tipo de servicio es?** MongoDB es una base de datos NoSQL orientada a documentos. Almacena datos en formato BSON (similar a JSON), lo que la hace flexible y escalable para manejar grandes volúmenes de datos no estructurados o semiestructurados.

   - **¿En qué tipo de sistema distribuido podría usarse?** Es ideal para aplicaciones web modernas, sistemas de gestión de contenido, análisis de datos en tiempo real y microservicios que requieren una base de datos flexible y de alto rendimiento. Puede formar parte de un clúster distribuido para asegurar alta disponibilidad y escalabilidad horizontal.

   - **¿Qué pasaría si ese servicio falla?** Si el contenedor de MongoDB falla, la aplicación que depende de él para almacenar y recuperar datos dejará de funcionar correctamente. Esto podría resultar en la imposibilidad de guardar nueva información, acceder a datos existentes o incluso en la caída completa de la aplicación si no hay mecanismos de redundancia o recuperación implementados.

   –

PARTE 5 – Análisis Crítico

1. **¿Cuál es la diferencia real entre imagen y contenedor?** La comparacion más clara es la de clase y objeto en programación. La **imagen** es la clase (el plano o la plantilla), que define todo lo necesario. El **contenedor** es el objeto (la instancia de esa clase), que es la entidad viva y en ejecución que consume recursos.

2. **¿Por qué Docker facilita la escalabilidad?** Porque permite lanzar nuevas copias de un servicio (contenedores) en segundos con un solo comando. Si una aplicación web necesita más capacidad, simplemente se ejecutan más contenedores de Nginx o del backend. Este proceso, conocido como **escalado horizontal**, es rápido y eficiente.

3. **¿Cómo se relaciona esto con microservicios?** Docker es la tecnología ideal para microservicios. Cada microservicio (ej: servicio de pagos, de usuarios, de notificaciones) se empaqueta en su propia imagen y se ejecuta en contenedores aislados. Esto permite que cada servicio se desarrolle, actualice y escale de forma independiente sin afectar a los demás.

4. **¿Qué ventajas ofrece frente a instalar software directamente?**

   - **Portabilidad:** Un contenedor funciona igual en la laptop de un desarrollador, en un servidor de pruebas o en la nube.
   - **Aislamiento:** Los contenedores no interfieren entre sí. Puedes tener diferentes versiones de Python o Java corriendo en la misma máquina sin conflictos.
   - **Consistencia:** Elimina el problema de “en mi máquina sí funciona”, ya que la imagen empaqueta el entorno completo.

- **Rapidez:** Crear y destruir contenedores es mucho más rápido que configurar máquinas virtuales o instalar software manualmente.

Parte 6 - Conclusión

Conclusión grupal

Esta práctica nos ha permitido entender de primera mano la potencia de Docker para la gestión de aplicaciones. Hemos visto cómo las imágenes actúan como plantillas inmutables y los contenedores como instancias ejecutables y aisladas, lo que simplifica enormemente el ciclo de vida del software. La capacidad de Docker para empaquetar aplicaciones con todas sus dependencias en unidades portátiles es clave para la escalabilidad y la implementación de arquitecturas modernas como los microservicios. Sin duda, Docker es una herramienta fundamental que nos ayuda a evitar los clásicos problemas de compatibilidad y a desplegar nuestras aplicaciones de forma más eficiente y confiable en cualquier entorno